



SET-Semesterprojekt

Sommersemester 2026

KI-gestützter Anamnese-Agent für die Hausarztpraxis

Dokumentation

Team 3: Vanessa Heil (3009551), My Melanie Hoang (3012722), Luisa Kolb (3008946),
Alina Lorenz (3013536), Mia Schatschneider (3014409)

Inhaltsverzeichnis

1. Einleitung und Projektziel.....	5
1.1 Ausgangssituation.....	5
1.2 Projektziel.....	5
1.3 Abgrenzung: Was leistet das System nicht?	6
2. Fachlicher, ethischer und rechtlicher Kontext	7
2.1 Medizinischer Kontext	7
2.2 Verwendete Leitlinien und fachliche Quellen	7
2.3 Medizinprodukterrechtliche und ethische Einordnung	8
3. Anforderungsanalyse.....	9
3.1 Funktionale Anforderungen (Muss/Soll/Kann).....	9
3.1.1 Patientenlisten-Import	9
3.1.2 Patientenidentifikation	9
3.1.3 Szenario Auswahl.....	10
3.1.4 Rollenerklärung und Zustimmung	10
3.1.5 Dialogsteuerung	10
3.1.6 Vitalparameter und Geräte.....	11
3.1.7 Red-Flag-Erkennung.....	11
3.1.8 Strukturierte Zusammenfassung	12
3.1.9 Interaktionsformen.....	12
3.1.10 Grafische Ausgabe und Avatar.....	12
3.1.11 Personalmodus	12
3.2 Nicht-funktionale Anforderungen (Muss/Soll/Kann).....	13
3.3 Use-Cases.....	14
4. Fachliches Konzept und Szenarien.....	15
4.1 Dialog- und Zustandsmodell.....	15
4.2 Szenario A – Akuter Husten/ Atemwegsinfekt/ Verdacht auf Pneumonie	16
4.3 Szenario B - Brustschmerz	16
4.4 Szenario C - Hypertonie-Kontrolle/ auffälliger Blutdruckwert.....	17
4.5 Szenario D - Typ-2-Diabetes/ metabolische Verlaufskontrolle	17
4.6 Red-Flag-Erkennung.....	17
4.7 Pulsoximeter-Auslösebedingungen	19
4.8 Konfigurationskonzept.....	19
5. Systemarchitektur.....	21
5.1 Gesamtübersicht über Systemarchitektur.....	21
5.2 Komponenten	22

5.3 Datenfluss.....	23
5.4 Geräteschnittstellen und Gerätesimulation	23
6. Implementierung.....	24
6.1 Verwendete Technologien	24
6.2 Implementierung des Dialogsystems	24
6.3 Implementierung der KI-Funktionalitäten.....	25
7. Qualitätssicherung und Tests.....	26
7.1 Teststrategie	26
7.2 Automatisierte Testergebnisse	26
7.3 Manuelle Prüfung und gefundene Fehler	27
7.3.1 Gerätesimulator ohne fachlichen Auslöser	27
7.3.2 Verzögerte statt unmittelbarer Eskalation.....	27
7.3.3 Verpflichtende Blutdruckeingabe im Diabetes-Szenario	28
7.3.4 Usability-Test mit externen Testpersonen	28
8. Einsatz von KI im Projekt	29
8.1 Verwendete KI-Tools.....	29
8.2 Typischer KI-gestützter Arbeitsablauf	29
8.3 Beispiel-Prompts und Wirkung.....	29
8.3.1 Fachliche Überarbeitung eines Szenarios.....	29
8.3.2 Aufbau eines neuen Diabetes-Szenarios	30
8.3.3 Überarbeitung vorhandener Szenarien einschließlich Tests	30
8.3.4 Fehlersuche anhand einer beobachtbaren Abweichung.....	30
8.3.5 Hilfestellung zum Verstehen des Dialogmodells.....	30
8.4 Prüfung KI-generierten Codes	31
8.5 Hilfreiche KI-Beiträge.....	31
8.6 Fehlerhafte oder riskante KI-Vorschläge.....	31
8.7 Grenzen und Reflexion	32
9. Projektmanagement und Teamarbeit.....	33
9.1 Projektorganisation	33
9.2 Aufgabenverteilung	33
9.3 Zusammenarbeit im Team	33
9.4 Herausforderungen im Projektverlauf.....	34
10. Installation und Bedienung.....	35
10.1 Voraussetzungen	35
10.2 Installation	35
10.3 Start	35

10.4 Optionaler KI-Chat	35
11. Einschränkungen und Ausblick	36
11.1 Bekannte Einschränkungen	36
11.2 Verbesserungsvorschläge für die Zukunft	36
11.2 Ausblick auf eine Praxisintegration	36
12. Fazit	37
13. Literaturverzeichnis	38

1. Einleitung und Projektziel

1.1 Ausgangssituation

Hausarztpraxen stehen zunehmend vor dem Problem, eine wachsende Anzahl an Patienten bei gleichzeitigem Personalmangel zu betreuen und behandeln. Durch die hohe Auslastung steht das medizinische Fachpersonal unter Druck, da nicht nur der medizinische Aspekt im Vordergrund steht, sondern auch organisatorische und dokumentationsbezogene Aufgaben bewältigt werden müssen.

Ein zentraler Bestandteil des Praxisalltags ist die medizinische Anamnese. Sie dient der strukturierten Erfassung von Symptomen, Vorerkrankungen und Risikofaktoren und bildet die Grundlage für die weitere medizinische Einschätzung. Die Durchführung einer vollständigen Anamnese ist jedoch zeitaufwendig und bindet wertvolle Ressourcen.

Gleichzeitig ist eine frühzeitige Erkennung kritischer Symptome von besonderer Bedeutung. Beschwerden wie Atemnot, Brustschmerzen oder auffällige Vitalwerte müssen möglichst früh identifiziert und priorisiert werden, um eine schnelle medizinische Versorgung sicherzustellen. Eine strukturierte digitale Voranamnese kann hierbei unterstützen, indem potenziell kritische Fälle bereits vor dem Arztkontakt erkannt und hervorgehoben werden [1].

1.2 Projektziel

Im Rahmen dieses Projekts wird daher ein KI-gestützter Anamnese-Agent entwickelt, der Patienten durch einen digitalen Frageprozess führt, relevante Informationen strukturiert erfasst und medizinisches Personal bei der Ersteinschätzung unterstützt.

Ziel des Systems ist die Unterstützung der Praxisabläufe durch Entlastung des medizinischen Personals, strukturierte Datenerfassung, Priorisierung kritischer Fälle mittels Red-Flag-Erkennung, sowie die Vorbereitung einer übersichtlichen Anamnesezusammenfassung für den Arzt.

Das System soll synthetische Patientendaten einlesen, die Identität einer ausgewählten Person prüfen, ein oder mehrere passende Szenarien aktivieren, die Rolle des Systems erklären, eine Zustimmung einholen und anschließend eine strukturierte Anamnese durchführen. Bei Fragen kann sich der Patient jederzeit an den Eulen-Avatar wenden. Messwerte können manuell angegeben oder durch Simulatoren erzeugt werden. Kritische Antworten werden durch deterministische Regeln erkannt. Abschließend entsteht eine gegliederte Zusammenfassung für ärztliches Personal.

Der Prototyp deckt die folgenden vier Standardszenarien ab [1], die sich mit Erkrankungen beziehungsweise medizinischen Anlässen befassen, die im Praxisalltag häufig vorkommen:

1. akuter Husten / Atemwegsinfekt / Verdacht auf Pneumonie,
2. Brustschmerz in der Hausarztpraxis,
3. Hypertonie-Kontrolle / auffälliger Blutdruckwert,
4. Typ-2-Diabetes / metabolische Verlaufskontrolle

Neben diesem beschriebenen „Patientenmodus“, der die KI-gestützte Anamnese enthält, existiert noch ein optionaler „Personalmodus“, der den Prototypen ergänzt, aber nicht Teil des eigentlichen Projektziels ist. Im Personalmodus kann das Praxispersonal neue Patienten anlegen, Termine vergeben und allen bestehenden Patienten eines oder mehrere der vier Standardszenarien zuordnen.

1.3 Abgrenzung: Was leistet das System nicht?

Das System ist ausdrücklich ein Assistenzsystem. Es darf zu keinem Zeitpunkt Diagnosen stellen, Therapieempfehlungen oder Entwarnung geben und ersetzt weder Untersuchung noch ärztliche Entscheidungen. Kritische Ausgaben sind Eskalationshinweise an das Praxispersonal, keine medizinische Endentscheidung. Nach Abschluss der Anamnese ist eine ärztliche Bewertung notwendig, da eine finale medizinische Einschätzung zwingend durch ärztliches Personal erfolgen muss.

2. Fachlicher, ethischer und rechtlicher Kontext

2.1 Medizinischer Kontext

Da hausärztliche Praxen unter hohem Zeitdruck stehen, soll der KI-gestützte Anamnese-Agent dabei helfen, medizinische Gesundheitsinformationen strukturiert zu erheben und Warnzeichen zu erkennen. Deshalb stehen dem Anamnese-Agenten vier Szenarien für die Anamnese zur Verfügung, die sich mit den häufig wiederkehrenden Beratungsanlässen befassen. In den Szenarien werden Beschwerden, Vorerkrankungen, Medikamente, Risikofaktoren und Warnzeichen erhoben. Die vier Szenarien werden in Kapitel 4 genauer erläutert.

Der Anamnese-Agent ist nicht für Diagnosen zuständig. Deshalb trennt das Projekt die strukturierte Erhebung von Gesundheitsinformationen von der medizinischen Bewertung dieser. Die Software dokumentiert Aussagen beziehungsweise Antworten des Patienten und Messwerte, prüft sich ergebende Warnzeichen explizit und übergibt das Ergebnis der Anamnese als strukturierte Zusammenfassung an ärztliches Personal. Die Verantwortung verbleibt durchgehend beim Menschen. Auch eine ausbleibende Red-Flag-Erkennung darf nicht als Ausschluss einer Erkrankung oder als Entwarnung verstanden werden.

2.2 Verwendete Leitlinien und fachliche Quellen

Das Projekt orientiert sich an den aktuellen hausärztlichen und versorgungsnahen Leitlinien, ohne diese vollständig in Software zu operationalisieren. Geeignete fachliche Bezugspunkte für die vier Szenarien sind insbesondere hausärztliche Leitlinien zu Brustschmerz, Husten, Hypertonie und Typ-2-Diabetes, zum Beispiel Leitlinien und Patienteninformationen von AWMF, DEGAM und den Nationalen Versorgungsleitlinien (siehe Tabelle unten) [2, 3, 4, 5, 6]. Für Brustschmerz ist insbesondere eine strukturierte Erfassung von Symptomen, Risikofaktoren und Warnzeichen relevant. Didaktisch kann der Marburger Herzscore als Beispiel für eine hausärztliche Entscheidungsregel betrachtet werden. Für Husten und Atemwegsbeschwerden stehen Symptombdauer, Fieber, Atemnot, Auswurf, thorakale Schmerzen, Vorerkrankungen und Warnzeichen im Vordergrund. Bei Hypertonie und Typ-2-Diabetes sind strukturierte Vitalparameter, Verlauf, Risikofaktoren, Medikamente, Adhärenz und Warnzeichen relevant.

Die fachlichen Quellen wurden nicht direkt und vollständig in Programmcode übersetzt. Die Leitlinien enthalten Empfehlungen für medizinisches Fachpersonal, Untersuchung, Diagnostik und Therapie. Für die strukturierte Erfassung medizinischer Informationen wurden nur geeignete Anamneseelemente, Warnzeichen und Verlaufparameter abgeleitet und in Alltagssprache übertragen, damit sie für den Patienten verständlich sind. Körperliche Untersuchungsbefunde, Labordiagnostik und therapeutische Entscheidungen sind nicht Teil des Prototyps.

Szenario	Grundlage	Verwendung im Projekt	Abgrenzung
A – Husten	DEGAM-Leitlinie „Husten“, Version 3.0, AWMF 053-013; ergänzend S3-Leitlinie zur ambulant erworbenen Pneumonie, AWMF 020-020	Symptombdauer, Hustenart, Auswurf, Dyspnoe, Thoraxschmerz, Allgemeinzustand, relevante Vorerkrankungen, Red-Flags und begründeter Pulsoximeter-Einsatz	Keine Diagnose „Pneumonie“, keine Therapieentscheidung
B – Brustschmerz	DEGAM-Leitlinie „Brustschmerz“, Kurzversion, Version 2.2, AWMF-Register-Nr. 053-023, überarbeitet 06/2024	Lokalisation, Beginn, Dauer, Charakter, Ausstrahlung, Belastungsbezug, Begleitsymptome, Risikofaktoren und didaktische Vorbereitung des Marburger Herzscores	Score nicht als Diagnose oder Entwarnung; körperliche Befunde müssen ärztlich bestätigt werden
C – Hypertonie	Nationale Versorgungsleitlinie Hypertonie, Kurzfassung, Version 1.0, AWMF nvl-009, 2023	Messbedingungen, Blutdruck, Puls, Gewicht, Beschwerden, Medikation und Adhärenz, Lebensstil, bekannte oder erstmals auffällige Werte	Keine Interpretation als Therapieindikation; Simulator ersetzt keine reale Messung
D – Typ-2-Diabetes	Nationale Versorgungsleitlinie Typ-2-Diabetes, Kurzfassung, Version 3.0, AWMF nvl-001, 2023	Verlauf, Symptome, Medikamente, Gewicht, Blutdruck, Vorbefunde, Folgeprobleme, Kontrolltermine und offene Fragen	Kein Blutzuckersimulator, keine Dosisanpassung und keine Therapieempfehlung

2.3 Medizinprodukterechtliche und ethische Einordnung

Der Prototyp ist nicht als einsatzfähiges Medizinprodukt konzipiert. Eine spätere Praxisintegration würde eine eigenständige regulatorische Bewertung erfordern [8], insbesondere hinsichtlich Zweckbestimmung, Risikoklasse, Qualitätsmanagement, klinischer Bewertung, Cybersicherheit und menschlicher Aufsicht. Ethisch relevant sind außerdem Fehlalarme, übersehene Warnzeichen, Automationsvertrauen, Verständlichkeit, Barrierefreiheit und die Gefahr, dass eine technisch formulierte Ausgabe fälschlich als ärztliche Bewertung verstanden wird.

3. Anforderungsanalyse

3.1 Funktionale Anforderungen (Muss/Soll/Kann)

Funktionale Anforderungen beschreiben die konkreten Funktionen und Abläufe des Systems, gemäß der Leitfrage „Welche Aufgaben muss das System erfüllen?“

Die funktionalen Anforderungen werden in drei Stufen priorisiert. Muss-Anforderungen müssen umgesetzt werden und enthalten in ihrer nachfolgenden Formulierung das Wort “muss”. Soll-Anforderungen sollen umgesetzt werden, da sie die Muss-Anforderungen ergänzen. Sie werden im nachfolgenden mit dem Wort “soll” formuliert. Kann-Anforderungen sind optional und können, müssen aber nicht zwingend umgesetzt werden.

3.1.1 Patientenlisten-Import

ID	Anforderung
F1	Der Import der Patientenliste muss über eine JSON-Datei über erfolgen.
F2	Das System muss beziehungsweise darf nur synthetische Patientendaten verwenden.
F3	Das System muss die Mindestfelder “Name, Vorname, Geburtsdatum” validieren.
F4	Bei ungültigem JSON oder fehlenden Pflichtfeldern muss das System eine verständliche Fehlermeldung ausgeben.
F5	Das System muss eine Freitext-Suche in der Tagesliste bereitstellen, die eine Filterung nach Namen, Vorname, Geburtsdatum und Patienten-ID im Personalmodus erlaubt.
F6	Das System soll optionale Felder robust behandeln.

3.1.2 Patientenidentifikation

ID	Anforderung
F7	Die Auswahl eines Patienten muss aus der Tagesliste erfolgen.
F8	Es muss eine Abfrage des Namens, Vornamens und Geburtsdatums des Patienten erfolgen.
F9	Das System muss eine erneute Prüfung bei Nichtübereinstimmung der Patientendaten durchführen und den Patienten darauf hinweisen, dass seine eingegebenen Daten nicht übereinstimmen/falsch sind (z.B. wenn er ein ungültiges Kalender- bzw. Geburtsdatum eingegeben hat).
F10	Bei wiederholter Fehlidentifikation (3 Versuche) soll das System das Praxispersonal darauf hinweisen.
F11	Das System protokolliert erfolgreiche und fehlgeschlagene Identifikationsversuche im Konsolenprotokoll.

3.1.3 Szenario Auswahl

ID	Anforderung
F12	Das System muss anzeigen, welche medizinischen Bereiche (Überbegriffe) die vier Szenarien A bis D abdecken.
F13	Die Auswahl des Szenarios muss aus den vier vorgegebenen Standardszenarien erfolgen.
F14	Das System muss die Zuordnung des passenden Szenarios anhand des geplanten Beratungsanlasses des Patienten ausführen oder F15.
F15	Der Patient muss das Szenario (oder auch mehrere Szenarien) manuell auswählen können.
F16	Das System muss jederzeit anzeigen, welches Szenario aktiv ist.
F17	Der Patient kann mehr als ein Szenario auswählen.

3.1.4 Rollenerklärung und Zustimmung

ID	Anforderung
F18	Das System muss dem Patienten verständlich erklären, dass es ausschließlich ein Assistenzsystem ist.
F19	Das System muss einen eindeutigen Hinweis ausgeben, dass keine Diagnose und keine Therapieempfehlung durch das System erfolgen.
F20	Das System muss eine einfache Zustimmung zur Durchführung der assistierten Anamnese einfordern.
F21	Der Patient muss die Möglichkeit des Abbruchs haben bzw. darf er seine Zustimmung verweigern.

3.1.5 Dialogsteuerung

ID	Anforderung
F22	Das System muss einen strukturierten Dialog mit definierten Zuständen durchführen.
F23	Das System muss für jedes Szenario alle Pflichtinformationen erfassen.
F24	Das System muss bei unklaren oder fehlenden Angaben Rückfragen stellen bzw. Hinweise einblenden.
F25	Das System muss den Dialog sauber und eindeutig abschließen.

F26	Das System soll dem Patienten kontextabhängige Folgefragen stellen und diese dynamisch ein- bzw. ausblenden.
F27	Das System soll zwischen freitextlichen Angaben und strukturierten Feldern trennen.

3.1.6 Vitalparameter und Geräte

ID	Anforderung
F28	Das System muss mindestens einen Gerätesimulator oder eine echte Geräteschnittstelle enthalten.
F29	Der Patient muss die Auswahl zwischen automatischer Messwertübernahme bei simulierten Vitalparametern oder manueller Eingabe haben.
F30	Das System muss plausible Wertebereiche validieren.
F31	Das System muss in der Zusammenfassung kennzeichnen, ob ein Wert/Vitalparameter gemessen, simuliert oder manuell angegeben wurde.
F32	Das System soll Simulatoren für Blutdruck, Gewicht und Pulsoxymetrie (Puls und Sauerstoffsättigung) enthalten.
F33	Das System soll Fehlerfälle wie Verbindungsabbruch oder ungültige Messwerte erkennen.

3.1.7 Red-Flag-Erkennung

ID	Anforderung
F34	Das System muss in jedem Szenario auf Basis von bestimmten Regeln definierte Warnzeichen erkennen.
F35	Das System muss bei kritischer Konstellation unmittelbar einen Eskalationshinweis geben.
F36	Bei kritischen Konstellation (Red-Flag-Severity = critical) muss die Anamnese abgebrochen und eskalier werden.
F37	Das System muss die ausgelösten Red-Flags protokollieren.
F38	Das System soll nachvollziehbar begründen, welche Eingabe(n) welche Red-Flag(s) ausgelöst haben.
F39	Das System soll unterscheidbare Eskalationsstufen besitzen, z.B. "ärztliche Prüfung erforderlich" und "sofortige ärztliche Übernahme erforderlich".

3.1.8 Strukturierte Zusammenfassung

ID	Anforderung
F40	Das System muss eine strukturierte Zusammenfassung für ärztliches Personal erstellen und ausgeben.
F41	Das System muss in der Zusammenfassung Patientenantworten, Messwerte, Red-Flags und offene Punkte voneinander trennen.
F42	Das System muss die strukturierte Zusammenfassung als JSON-Datei exportieren.
F43	Das System soll den Export als PDF oder HTML zusätzlich ermöglichen.
F44	Das System soll in der Zusammenfassung synthetische Daten klar und eindeutig markieren.

3.1.9 Interaktionsformen

ID	Anforderung
F45	Das System muss über eine Bildschirmausgabe mit Tastatureingabe funktionieren.
F46	Das System soll Sprachinteraktion für die Ein- und/oder Ausgabe nutzen.
F47	Das System soll einen stabilen Fallback auf die Tastatureingabe besitzen.

3.1.10 Grafische Ausgabe und Avatar

ID	Anforderung
F48	Das System muss eine grafische Ausgabe besitzen.
F49	Das System muss mindestens drei unterscheidbare Gesprächszustände besitzen.
F50	Das System muss einen einfachen abstrakten Avatar besitzen.
F51	Das System/der Avatar soll mindestens vier verschiedene Zustände besitzen, z.B. für das Begrüßen, Zuhören, Nachdenken, Sprechen, eine Warnhinweis-Erkennung oder den Abschluss der Anamnese.
F52	Das System soll über einfache Animationen oder Zustandswechsel verfügen.

3.1.11 Personalmodus

ID	Anforderung
F53	Das System soll neben dem Patientenmodus noch einen Personalmodus besitzen.

F54	Praxispersonal soll neue Patienten mit den wichtigsten Informationen anlegen können.
F55	Neu angelegten Patienten kann ein Termin und ein oder mehrere Szenarien zugeordnet werden.

3.2 Nicht-funktionale Anforderungen (Muss/Soll/Kann)

Nicht-funktionale Anforderungen beschreiben die Qualitätsmerkmale des Systems und richten sich nach der Leitfrage „Wie soll das System arbeiten?“. Nicht-funktionale Anforderungen beziehen sich auf die Aspekte Sicherheit, Datenschutz, Performance, Modularität, Robustheit, Nachvollziehbarkeit, Testbarkeit, Benutzerfreundlichkeit, Zuverlässigkeit, Wartbarkeit, Datensparsamkeit und Reproduzierbarkeit.

Die Priorisierung der nicht-funktionalen Anforderungen in Muss-, Soll- und Kann-Anforderungen erfolgt nach dem gleichen Schema wie unter Abschnitt 3.1 erklärt.

ID	Anforderung
NF1	Die Red-Flag-Erkennung beziehungsweise Red-Flag-Entscheidungen müssen nachvollziehbar und mit Quellen belegt sein.
NF2	Konfigurationsentscheidungen und Ausgaben müssen nachvollziehbar sein.
NF3	Medizinische Logik, Dialogzustände, Import und Gerätesimulatoren müssen ohne UI automatisiert testbar sein.
NF4	UI, Dialoglogik, Red-Flag-Regeln, Gerätezugriff und Datenimport müssen getrennt sein.
NF5	Ungültige Eingaben, fehlende Messwerte, Gerätefehler und Abbrüche müssen kontrolliert behandelt werden.
NF6	Die Interaktion soll klar, freundlich, langsam genug und ohne Fachjargon erfolgen.
NF7	Regeln, Szenarien und Geräteoptionen sollen konfigurierbar oder leicht erweiterbar sein.
NF8	Es dürfen/müssen nur Daten erhoben werden, die für das jeweilige Szenario und die Demonstration erforderlich sind.
NF9	Das System darf keine (muss keine) medizinisch riskanten Aussagen, Diagnosen, Entwarnungen oder Therapieempfehlungen generieren.
NF10	Demo-Szenarien, Testdaten und Konfigurationen müssen reproduzierbar sein.
NF11	Das System darf nur (muss) synthetische Patientendaten verwenden. Reale Patientendaten dürfen nicht verwendet werden.

NF12	Das System soll Antworten innerhalb von 5-10 Sekunden generieren.
NF13	Das System soll 99,5% der Zeit verfügbar sein.
NF14	Die Oberfläche und die gestellten Anamnese-Fragen müssen auch für medizinische Laien verständlich sein.
NF15	Der KI-Agent soll auf die Patientenakte Bezug nehmen.

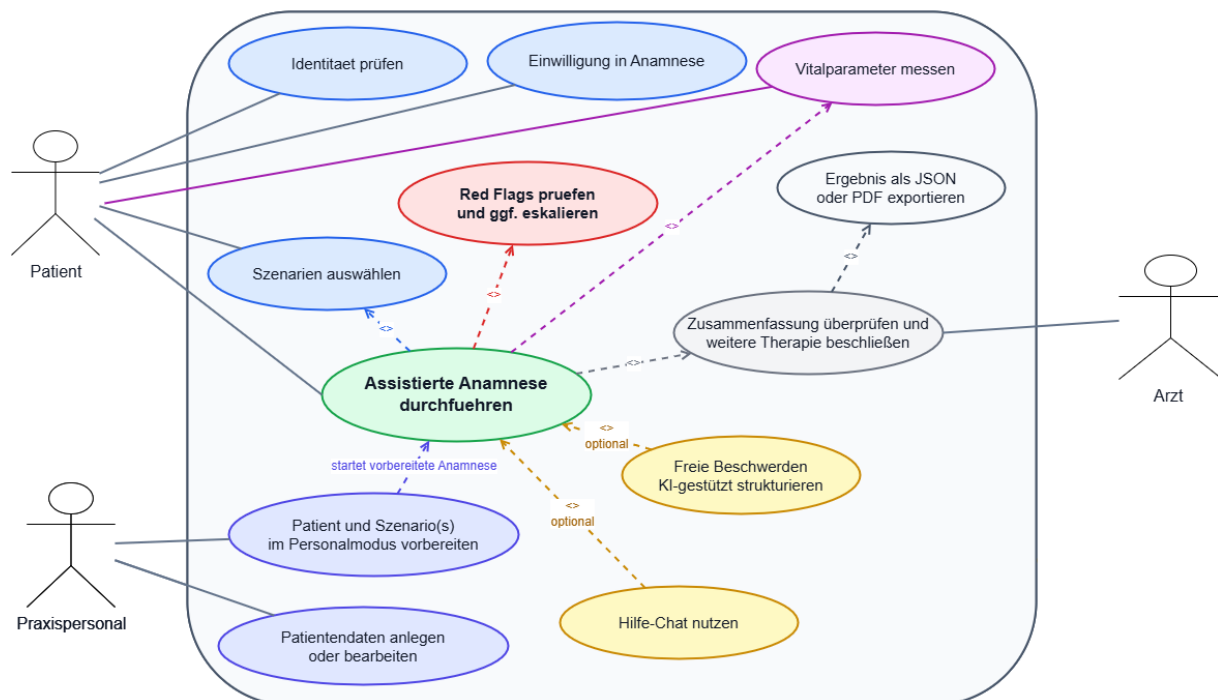
3.3 Use-Cases

Das Use-Case-Diagramm zeigt die funktionalen Anforderungen des KI-gestützten Anamnese-Agenten. Akteure, die mit dem System interagieren sind der Patient und der Arzt beziehungsweise das Praxispersonal. Ergänzend kommen Geräte beziehungsweise Simulatoren und der KI-Assistent hinzu.

Der Patient kann folgendes ausführen: Er meldet sich an, wählt ein oder mehrere Szenarien aus, stimmt der Anamnese zu (oder lehnt diese ab), wählt aus, ob er die Anamnese als geführtes Gespräch oder im Formularmodus durchführen möchte, beschreibt optional seine aktuellen Symptome im Chat, beantwortet die Anamnese-Fragen, misst oder simuliert Vitalparameter und prüft die Zusammenfassung. Optional kann er Fragen an die KI im Hilfe-Chat stellen.

Der Arzt erhält die strukturierte Zusammenfassung der Anamnese für die ärztliche Befundung. Das Praxispersonal kann die Patienten-Liste einsehen und Patienten und Szenarien im Personalmodus vorbereiten, wird bei Eskalation oder kritischen Angaben informiert und

UML-Use-Case-Diagramm - KI-gestützter Anamnese-Agent



4. Fachliches Konzept und Szenarien

4.1 Dialog- und Zustandsmodell

Der Dialog beziehungsweise die Interaktion zwischen dem System (KI-gestützter Anamnese-Agent) und dem Benutzer (Patient) wird durch das Dialogmodell veranschaulicht. Es werden verschiedene Zustände (States) beschrieben, in denen sich der Dialog gerade befindet, und Übergänge (Transitions) deutlich, wenn der Dialog zu bestimmten Zeitpunkten zwischen Zuständen wechselt. Hinzu kommen Aktionen, die in den jeweiligen Zuständen stattfinden und Bedingungen, die dazu führen, dass verschiedene Aktionen oder Zustände ausgelöst werden.

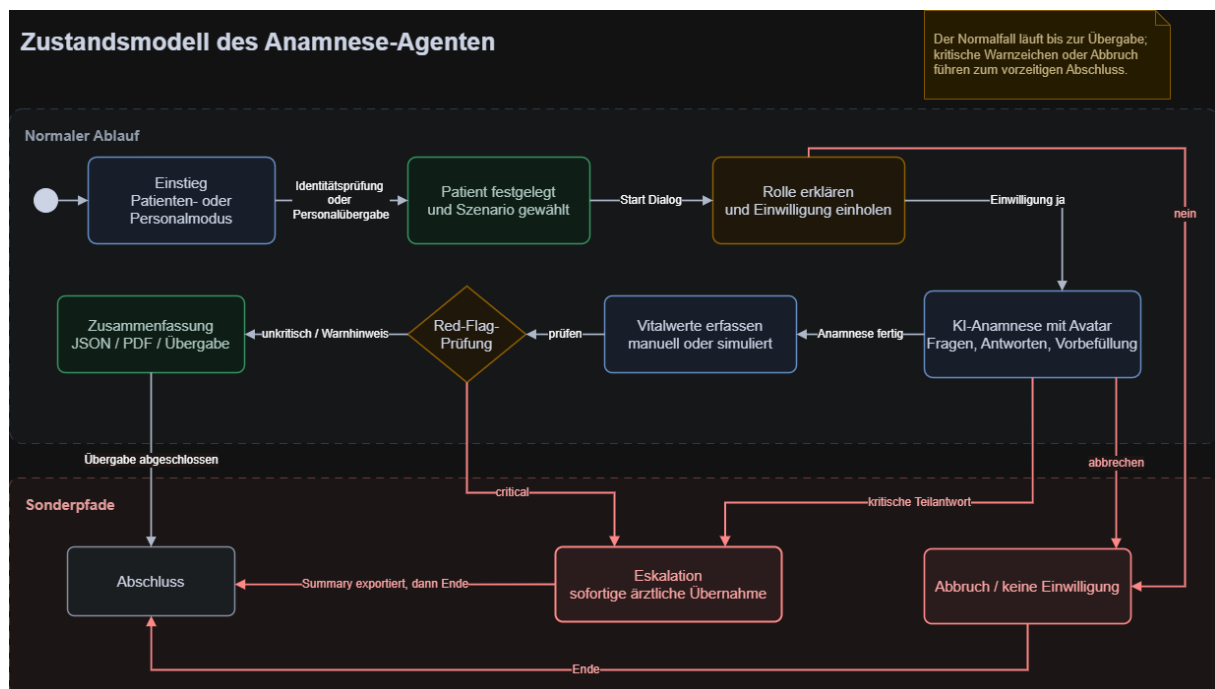
In diesem Projekt ist das Dialogmodell ein endlicher Zustandsautomat (State Machine), der in der Datei `app/dialogue/state_machine.py` implementiert ist. Der `DialogueController` in `app/dialogue/dialogue_controller.py` steuert den Dialog und entscheidet, wann welche Frage sichtbar ist beziehungsweise wann entsprechende Folgefragen gestellt werden.

Der KI-gestützte Anamnese-Agent dient der strukturierten Erfassung relevanter, medizinischer Informationen. Der Patient wird aufgefordert, dem Anamnese-Prozess und der Erfassung seiner Daten zuzustimmen. Nach erfolgreicher Zustimmung wird der Patient zur eigentlichen Anamnese weitergeleitet. Der Patient füllt für die Anamnese entweder einen Fragebogen aus oder wird durch einen sprach- und dialogbasierten Frageprozess geführt. Das System analysiert die Antworten des Patienten und generiert in einigen Fällen kontextabhängige Folgefragen. Potenzielle Risikofaktoren (Red Flags) werden während der Anamnese erkannt, verarbeitet und das System reagiert anschließend entsprechend. Bei kritischen Warnzeichen, die das Eingreifen des Praxispersonals oder eines Arztes erfordern, wird die Anamnese vorzeitig abgebrochen und ein entsprechender Hinweis angezeigt. Der Dialog wird beendet, sobald alle relevanten Informationen erhoben und alle Pflichtfragen beantwortet wurden oder der Patient die Anamnese abbricht. Nach erfolgreicher Beendigung der Anamnese hat der Patient die Möglichkeit, seine Antworten noch einmal zu bearbeiten.

Die Zustände (Zustandsmodell), die während der Anamnese durchlaufen werden, sind in der nachfolgenden Tabelle dargestellt:

Zustand	Beschreibung
EXPLAIN_ROLE	System erklärt seine Rolle als Anamnese-Assistent. Im Controller wird dieser Zustand direkt weitergeführt.
REQUEST_CONSENT	Patient stimmt der KI-gestützten Anamnese zu. Bei Ablehnung springt Dialog zu END.
ANAMNESIS	Hauptdialog: geführte oder formularbasierte Anamnese. Kritische Teilantworten führen direkt zu ESCALATION.
VITAL_PARAMETERS	Vitalparameter werden aus Eingabe/ Simulatoren übernommen.
RED_FLAG_CHECK	Automatische Prüfung der Antworten auf kritische Symptome, Warnhinweise
ESCALATION	Bei Red-Flags: Eskalation zum ärztlichen Personal
SUMMARY	Zusammenfassung der Anamneseangaben
HANDOVER	Übergabe der Ergebnisse an das Personal
END	Abschluss des Dialogs

Die Übergänge und Bedingungen der verschiedenen Zustände sind im nachfolgenden Diagramm dargestellt:



4.2 Szenario A – Akuter Husten/ Atemwegsinfekt/ Verdacht auf Pneumonie

Szenario A befasst sich mit dem Krankheitsbild des akuten Hustens beziehungsweise dem Atemwegsinfekt und dem Verdacht auf Pneumonie. Erfasst werden Dauer und Verlauf, Hustenart, Auswurf und Farbe, Blutbeimengung, Fieber, Temperatur, Schüttelfrost, Atemnot, Belastungsdyspnoe, Atemnot in Ruhe, Beeinträchtigung beim Sprechen, Zyanose, Atemgeräusche, thorakale Beschwerden, Allgemeinzustand, Verwirrtheit, Ohnmacht, Vorerkrankungen, Risikofaktoren und Medikamente. Folgefragen erscheinen nur, wenn sie durch eine Vorantwort begründet sind, beispielsweise wird die Schleimfarbe nur nach bejahtem Auswurf abgefragt.

Zu den kritischen Warnzeichen gehören unter anderem Atemnot in Ruhe, notwendiges Luftholen beim Sprechen, Zyanose, Verwirrtheit, Ohnmacht, Blut beim Husten, rasche Verschlechterung und ein stark reduzierter Allgemeinzustand. Allgemeine Kurzatmigkeit löst nicht automatisch den sofortigen Abbruch aus, weil zunächst noch relevante Differenzierungsfragen beantwortet werden müssen [2, 3].

4.3 Szenario B- Brustschmerz

Das Szenario B deckt den Bereich Brustschmerz ab. Die Anamnese erfasst Lokalisation, Beginn, Dauer, aktuelle Stärke, Schmerzcharakter, Ausstrahlung und Belastungsabhängigkeit des Brustschmerzes, Besserung in Ruhe, Einfluss von Bewegung, Atmung, Husten, Druck, Essen oder Schlucken sowie Atemnot, Kaltschweißigkeit, Übelkeit, Synkope, Verwirrtheit, Schwäche und Verschlechterung.

Bekannte Herz-Kreislauf-Erkrankungen, Risikofaktoren und Medikamente werden getrennt dokumentiert und teilweise aus der vorhandenen Patientenakte entnommen.

Brustschmerz erfordert eine besonders konservative Sicherheitsstrategie. Kritische Kombinationen (Red-Flags) führen zum sofortigen Abbruch der Anamnese und zur Sichtung des Patienten durch das Praxispersonal oder einen Arzt. Es wird keine Entwarnung formuliert. Der Marburger-Herzscore wird ausschließlich didaktisch vorbereitet [4].

4.4 Szenario C- Hypertonie-Kontrolle/ auffälliger Blutdruckwert

In Szenario C werden zwei Ausgangssituationen unterschieden: eine bekannte Hypertonie mit Verlaufskontrolle und einen erstmals auffälligen Wert. Im ersten Zweig werden letzte Kontrolle, Behandlungsänderung und Werte, die zu Hause gemessen wurden, erhoben; im zweiten Zweig Zeitpunkt, Ort, Wiederholungsmessung und frühere Auffälligkeiten. Gemeinsam erfasst werden Messbedingungen, Blutdruck, Puls, Gewicht, Brustschmerz, Atemnot, neurologische Symptome, Sehprobleme, Kopfschmerz, Ohnmacht, Herzklopfen, Vorerkrankungen, Medikation, Adhärenz und Lebensstil [5].

Blutdruck, Puls und Gewicht können manuell beziehungsweise mündlich angegeben oder simuliert werden. Kritische Beschwerden und kritische Messwerte (Red-Flags) werden unmittelbar geprüft und das System reagiert entsprechend (z.B. Abbruch der Anamnese und Übergabe an medizinisches Personal). Das System bezeichnet auffällige Werte jedoch nicht als Diagnose und leitet auch keine Behandlung ab.

4.5 Szenario D- Typ-2-Diabetes/ metabolische Verlaufskontrolle

Das Diabetes-Szenario wurde als Verlaufskontrolle und nicht als allgemeine Akutdiagnostik umgesetzt. Es erfasst Allgemeinbefinden, Hinweise auf Unter- oder Überzuckerung, Gewicht und Gewichtsveränderungen, Blutdruck, bekannte Diagnosen, Medikamente und deren Einnahme, Lebensstil, mögliche Folgeprobleme an Augen, Nieren, Nerven, Füßen, Herz und Kreislauf, Fußwunden, Kontrolltermine, bekannte HbA1c- beziehungsweise Blutzuckerangaben und offene Fragen [6].

Ein Blutzuckersimulator ist in der Aufgabenstellung nicht gefordert und wurde deswegen nicht implementiert. Blutzuckerwerte müssen deshalb vom Patienten eingegeben werden. Wenn Blutdruck- und/oder Gewichtswerte als „unbekannt“ markiert werden, wird anschließend eine Messung beziehungsweise Simulation dieser Werte eingefordert. Die Zusammenfassung gruppiert die Angaben in Verlauf, aktuelle Symptome, Medikation, Komplikationen, Vorbefunde und offene Fragen.

4.6 Red-Flag-Erkennung

Die Red-Flag-Erkennung ist getrennt von der Benutzeroberfläche in einer eigenständigen Regelkomponente umgesetzt. Sie erhält strukturierte Antworten und vorhandene Messwerte aus der Anamnese und gibt ausgelöste Warnzeichen mit Regelnummer, Beschreibung, Schweregrad und Grund der Auslösung zurück. Die Stufe „warning“ kennzeichnet eine erforderliche ärztliche Prüfung; „critical“ führt zum Abbruch der Anamnese und zum Hinweis „Bitte sofort dem Praxisteam melden“. Dadurch bleiben die Entscheidungen nachvollziehbar und automatisiert testbar.

In den nachfolgenden Tabellen sind Beispiele für kritische Auslöser und Warnhinweise für alle vier Szenarien sowie die zugrundeliegenden numerischen Grenzwerte der Simulatoren dargestellt [2, 3, 4, 5, 6].

Szenario	Beispiele kritischer Auslöser	Beispiele für Warnhinweise
Husten	Blut beim Husten, Atemnot in Ruhe/ beim Sprechen, Sprechbeeinträchtigung, Zyanose, Verwirrtheit, Ohnmacht, schwere Verschlechterung, Atemfrequenz $\geq 30/\text{min}$, Fieber $\geq 40^\circ\text{C}$, Rauch/Reizstoffe, auffälliges Atemgeräusch	Kurzatmigkeit, $39^\circ\text{C} \leq \text{Fieber} \leq 40^\circ\text{C}$, thorakale Schmerzen, Vorerkrankungen wie COPD, Asthma, Herzinsuffizienz, Belastungsdyspnoe
Brustschmerz	Aktuell starke oder anhaltende Schmerzen, Atemnot (in Ruhe), Kaltschweißigkeit, ausgeprägte Schwäche, neurologische Auffälligkeit	Bekannte KHK, Alter > 55 plus Risikofaktoren wie Rauchen, Diabetes, Bluthochdruck, Hypercholesterinämie oder familiäre Belastung
Hypertonie	Brustschmerz, Atemnot, neurologische Symptome, Ohnmacht, akute Sehprobleme oder Synkope	Sehstörung/ Kopfschmerz ohne systolischen Blutdruck ≥ 180 , systolischer Blutdruck 160-179 mmHg, diastolischer Blutdruck 100-119 mmHg
Diabetes	Bewusstseinsstörung, schwere Symptomkonstellation, Brustschmerz, Atemnot, Blutzucker $\geq 300 \text{ mg/dl}$, kritischer Blutdruck	Hinweise auf Unter-/Überzuckerung, plötzliche Sehverschlechterung, Fußprobleme/ offene Wunden ohne kritische Verschlechterungszeichen

Parameter	Kritischer Wert	Betroffene Szenarien
Systolischer Blutdruck	$\geq 180 \text{ mmHg}$	Brustschmerz
		Diabetes
		Hypertonie
Diastolischer Blutdruck	$\geq 120 \text{ mmHg}$	Brustschmerz
		Diabetes
		Hypertonie

SpO ₂	< 92 %	Brustschmerz
		Husten
Puls	≥ 120/min	Hypertonie
		Husten

4.7 Pulsoximeter-Auslösebedingungen

Das Pulsoximeter wird in diesem Szenario nicht routinemäßig gestartet und läuft auch nicht unbemerkt im Hintergrund. Eine Messung wird nur bei respiratorischen Beschwerden oder weiteren medizinisch begründeten Risikohinweisen angeboten, beispielsweise bei Kurzatmigkeit, Brustbeschwerden, hohem Fieber, einem stark eingeschränkten Allgemeinzustand oder einer bekannten chronischen Lungenerkrankung [2, 3]. Gewicht und Blutdruck werden in diesem Szenario hingegen nicht ohne medizinischen Anlass simuliert.

In der aktuellen Implementierung führen jedoch viele dieser Voraussetzungen bereits unmittelbar zu einer kritischen Red-Flag-Erkennung. Dadurch wird die Anamnese häufig beendet, bevor eine Messung mit dem Pulsoximeter durchgeführt werden kann. Erfolgt dennoch eine Messung, werden Sauerstoffsättigung und Puls eindeutig als simulierte Messwerte gekennzeichnet.

4.8 Konfigurationskonzept

Das Konfigurationskonzept beschreibt, welche Bestandteile des Systems ohne Programmierung angepasst werden können, an welcher Stelle die Konfiguration hinterlegt ist und wer sie anpassen darf.

In diesem Projekt wird jedoch eine Code-nahe Konfiguration verfolgt, da die Hauptbestandteile des Systems (Szenarien, Regeln, Red-Flag-Erkennung) in Python-Datenstrukturen umgesetzt sind. Die nachfolgend dargestellten Konfigurationsbereiche beziehen sich deshalb auf Python-Dateien. Eine Trennung von Code und Konfiguration ist nicht umgesetzt, wäre aber für eine eventuelle Weiterentwicklung des Systems zu einem späteren Zeitpunkt sinnvoll.

Konfiguration	Beschreibung	Ablageort	Konfigurierbar von
Szenario-Definitionen	Jedes medizinische Szenario definiert seine eigenen Fragen mit Text, Eingabetyp, Pflichtfeld-Status und Slider-Grenzen	app/scenarios/*.py	Entwickler, medizinisches Fachpersonal
Patientendaten	Externe Patientendaten (Diagnosen, Risikofaktoren, Medikation) steuern, welche Fragen automatisch beantwortet werden und welche Folgefragen erscheinen	app/patient_import/patient_schema.py (Schema) und externe JSON-Datei	medizinisches Fachpersonal

Red-Flag-Regeln	Bestimmte Antwortmuster lösen eine Eskalation aus. Die Regeln sind als Python-Logik in der RedFlagEngine hinterlegt	app/red_flag/red_flag_engine.py	Entwickler, medizinisches Fachpersonal
Dialog-Steuerungslogik	Die is_question_visible-Methode bestimmt szenariospezifisch, welche Fragen unter welchen Bedingungen angezeigt werden	app/dialogue/dialogue_controller.py	Entwickler
Simulations-Parameter	Die Vitalparameter-Simulation (Puls, Blutdruck, Gewicht) verwendet hinterlegte BMI-Formeln und Wertebereiche	app/devices/simulator_s.py	Entwickler

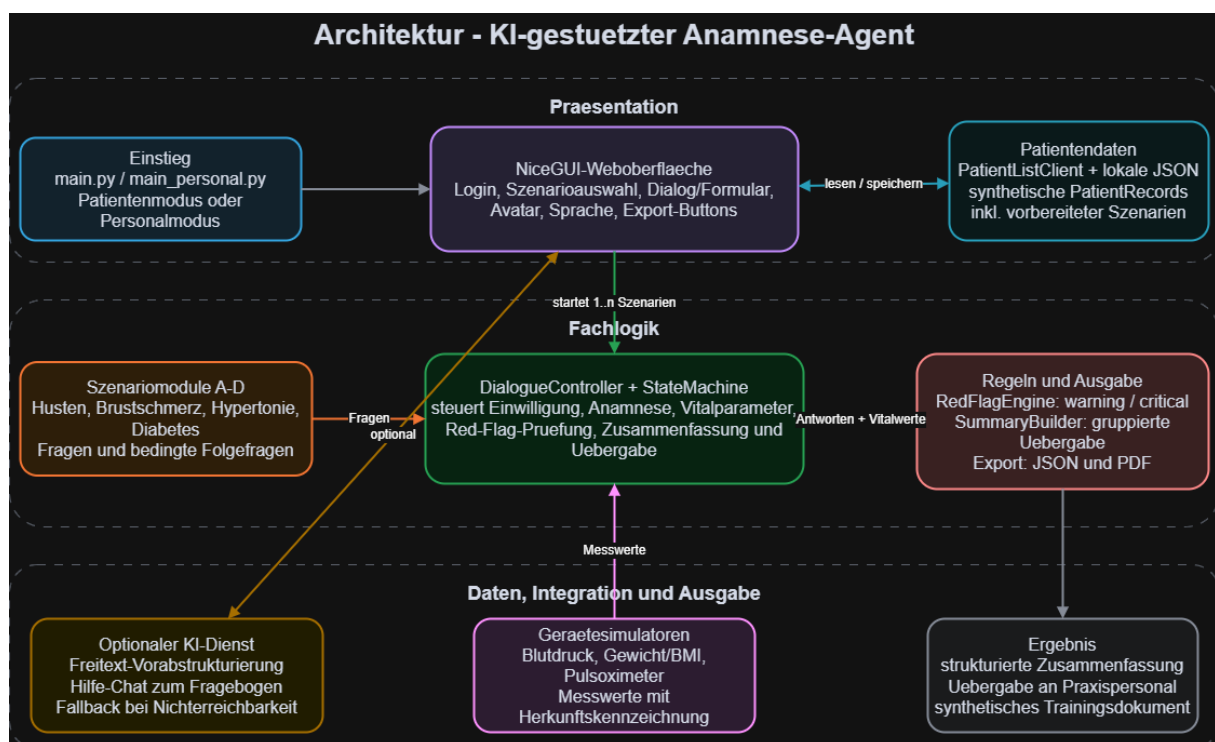
5. Systemarchitektur

Dieses Kapitel beschreibt den technischen Aufbau des KI-gestützten Anamnese-Agenten.

5.1 Gesamtübersicht über Systemarchitektur

Der Ablauf der Patientendaten-Erfassung sowie der Anamnese mit anschließender Zusammenfassung wird von einem KI-gestützten System ermöglicht, das aus mehreren Komponenten besteht. Die Hauptbestandteile des Systems sind in Abschnitt 5.2 genauer beschrieben.

Das System folgt einer Schichtenarchitektur. Die Architektur trennt Präsentation, Dialogsteuerung, fachliche Szenarien, Red-Flag-Erkennung, Gerätezugriff, Datenimport und Ausgabe. Diese Trennung ermöglicht es, die medizinisch relevante Logik ohne Browseroberfläche zu testen. In der nachfolgenden Abbildung sind die drei Hauptschichten des Systems erkennbar: Präsentation, Fachlogik und Daten.



Präsentationsschicht: Die NiceGUI-Weboberfläche stellt die Anamnese-Fragen dar und enthält den Eulen-Avatar. Die UI kommuniziert ausschließlich über den Dialogue-Controller.

Fachlogik: Der DialogueController steuert den gesamten Anamnese-Ablauf und lädt die Fragen für die vier Szenarien, führt die Red-Flag-Erkennung durch und erzeugt die Zusammenfassung der Anamnese.

Daten: Zu den Daten zählt die strukturierte Zusammenfassung der Anamnese sowie der KI-Dienst, der in die Weboberfläche integriert ist. Zusätzlich stehen Gerätesimulatoren zur Verfügung, die Messwerte für die Anamnese erzeugen.

5.2 Komponenten

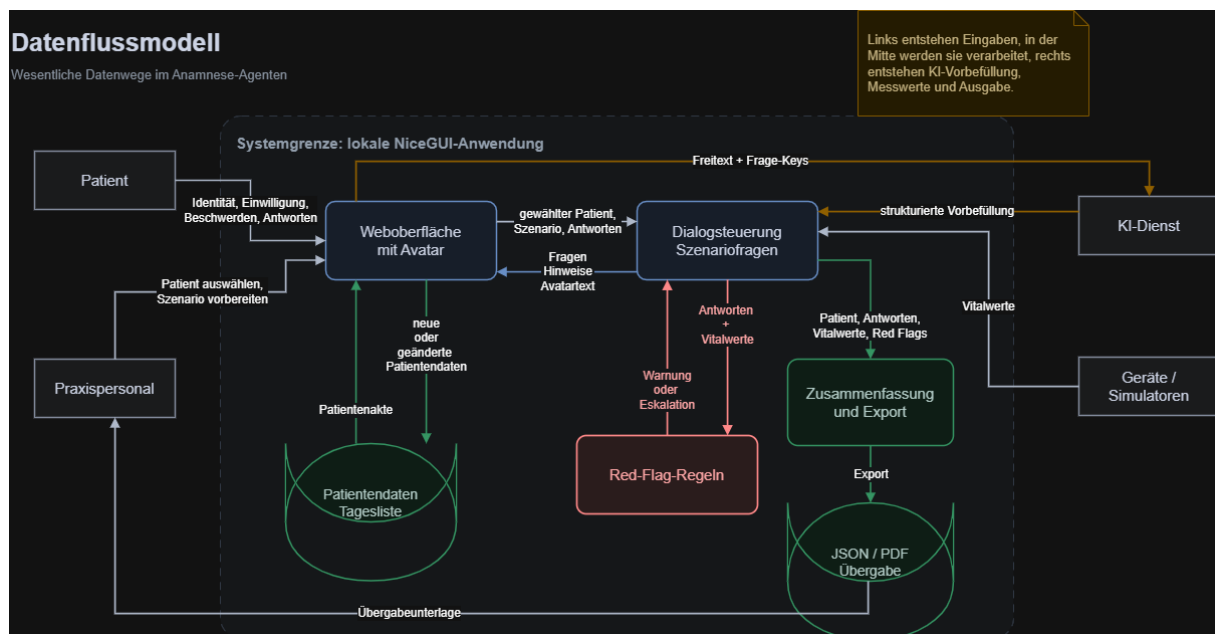
Komponente	Aufgabe	Wichtige Dateien
Weboberfläche	Login, Personalmodus, Szenarioauswahl, Zustimmung, Dialog/Formular, Avatar, Sprache, Bearbeitung, Export	app/ui/web_app.py, main.py, main_personal.py
Import und Identität	Lokales JSON parsen, Patientendatensätze normalisieren, Identität prüfen	patient_list_client.py, patient_schema.py, identity_check.py
Dialogsteuerung	Zustände, Pflichtfragen, bedingte Sichtbarkeit, Eingabevalidierung, Live-Eskalation	dialogue_controller.py, state_machine.py, consent_flow.py
Szenarien	Fragenlisten und szenariospezifische Hilfslogik	app/scenarios/*.py
Medizinische Regeln	Deterministische Red-Flags und Szenarioempfehlung	red_flag_engine.py, scenario_recommendation.py
Geräte	Zufallsbasierte Simulation von Blutdruck, Gewicht, Puls und SpO ₂ ; Adapterstruktur für spätere Geräte	simulators.py, Adapterdateien
KI-Extraktor	Optionale Strukturierung einer freien Beschwerdeschilderung über DeepSeek.	ai/symptom_extractor.py
Ausgabe	Gruppierte Zusammenfassung sowie JSON- und PDF-Ausgabe	summary_builder.py, export_json.py, export_pdf.py
Logging	Zustandswechsel, Warnungen und Eskalationen auf der Konsole	audit_logger.py
Tests	Unit- und Integrationstests für Dialogsteuerung, Red-Flags, Szenarien, Import, Export und UI-Formularhilfsfunktionen	app/tests/unit/*.py

5.3 Datenfluss

Der Datenfluss beschreibt, wie die Daten durch das System wandern. Dabei kann unterteilt werden, woher die Daten kommen (Quellen, Eingänge), wie sie verarbeitet (Transformation) und anschließend gespeichert (Ausgänge) werden.

In diesem Projekt existieren verschiedene Datenquellen. Über die synthetische Patientenakte erhält das System wichtige Patientendaten. Die wichtigsten Daten für das System gibt der Patient beim Beantworten der Anamnese-Fragen über die UI ein. Zusätzlich generieren die verschiedenen Simulatoren Vitalparameterwerte für Puls, Blutdruck, Gewicht und Sauerstoffsättigung.

Der Datenfluss beziehungsweise die Datenverarbeitung ist im nachfolgenden Diagramm dargestellt.



Es gibt keine Datenbank, in der die Daten gespeichert werden, stattdessen werden die Daten während einer Session gehalten und gehen verloren, sobald diese beendet wird. Es ist jedoch möglich, die strukturierte Zusammenfassung der Anamnese mit den Patientenantworten, Vitalparametern und erkannten Red-Flags als pdf-Datei zu speichern.

5.4 Geräteschnittstellen und Gerätesimulation

Der KI-gestützte Anamnese-Agent besitzt zum jetzigen Zeitpunkt keine echte Geräteschnittstelle. Stattdessen sind in den vier Szenarien an den entsprechenden Stellen der Anamnese Gerätesimulatoren verfügbar, die Vitalparameter für Blutdruck, Puls, Gewicht und Sauerstoffsättigung simulieren. Neben dieser simulierten Erzeugung von Vitalparametern hat der Patient auch die Möglichkeit, die Werte manuell einzugeben.

Für eine spätere Anbindung von echten Geräten zum Messen von Vitalparametern ist die Klasse „DeviceInterface“ in der Datei `app/devices/device_interface.py` vorgesehen.

6. Implementierung

6.1 Verwendete Technologien

Die nachfolgende Tabelle stellt die in diesem Projekt verwendeten Technologien übersichtlich dar.

Technologie	Verwendung
Python 3.12	Gesamte Anwendungs-, Fach- und Testlogik
NiceGUI	Lokale Weboberfläche auf Basis eines Python-Backends und Browser-Frontends
pytest	Unit- und Integrationstests
ReportLab	Erzeugung der PDF-Zusammenfassung
Browser APIs Speech	Sprachausgabe über speechSynthesis und Mikrofoneingabe über Browser-Spracherkennung
DeepSeek V4 Pro	Optionales Vorfüllen passender Anamnese-Fragen über Freitexteingabe
JSON	Patientenliste und maschinenlesbarer Zusammenfassungsexport
SVG	Darstellung des Eulen-Avatars
CSS	Zustandsgestaltung und Animationen des Avatars

6.2 Implementierung des Dialogsystems

Die UI bietet einen Patienten- und einen Personalmodus. Der Personalmodus läuft standardmäßig auf Port 8081 und erlaubt die Auswahl von Patienten, die Ergänzung von Kurzangaben beziehungsweise wichtigen persönlichen Daten zu den Patienten und die Vorbereitung mehrerer Szenarien. Der Patientenmodus läuft auf Port 8080. Hier kann der Patient das passende oder bereits bevorzugte Szenario auswählen. Bei der Beantwortung der Anamnesefragen hat er die Wahl zwischen einem geführten Gespräch im Dialogmodus mit Sprachausgabe und einem Formularmodus. Im geführten Modus liest die Browser-Sprachausgabe die Anamnesefragen optional vor. Über eine Mikrofontaste kann das aktuelle Textfeld per Spracheingabe ausgefüllt werden. Es ist jederzeit möglich, zwischen Formularmodus und der sprachbasierten, geführten Anamnese zu wechseln. Da Browser und Betriebssystem unterschiedliche Speech-APIs und Stimmen bereitstellen, bleibt die Tastatur der verlässliche Fallback.

Im Allgemeinen unterstützt der Dialogcontroller einen schrittweisen Modus und die Übergabe eines vollständigen Antwortsatzes aus dem Formular. Pflichtfelder werden am Ende der Anamnese geprüft und müssen, wenn sie nicht ausgefüllt wurden, noch ergänzt werden. Bei Fragen, die numerischen Angaben als Antwort erwarten (v.a. bei Vitalparametern), ist es möglich und auch zulässig,

„unbekannt“ auszuwählen. Ist dies der Fall, werden Folgefragen bzw. Felder eingeblendet, die den Patienten auffordern, die numerischen Angaben zu messen oder simulieren zu lassen.

6.3 Implementierung der KI-Funktionalitäten

Die KI-Funktionalität wurde mithilfe der von der Hochschule zur Verfügung gestellten API implementiert. Die API verbindet den Agenten direkt mit dem lokalen LLM der Hochschule. Um eine Verbindung herzustellen, muss man sich entweder im VPN der Hochschule befinden oder mit dem Netzwerk vor Ort verbunden sein.

Die KI ist im Assistenten-Chat des Eulen-Avatars integriert. Sie erhält als Eingaben die Fragen des Patienten zu einem Begriff oder einer Formularfrage. Zusätzlich werden alle Fragen des Szenarios, aktuelle Antworten, der Chat-Verlauf und die Patientenakte übergeben. Dadurch kann der KI-Assistent Begriffe und Fragen klären und sich gezielt auf den jeweiligen Patienten beziehen.

Die medizinische Bewertung erfolgt jedoch nicht durch den KI-Assistenten. Insbesondere die Red-Flag-Erkennung und sämtliche Eskalationsentscheidungen werden ausschließlich durch deterministische Regeln getroffen.

7. Qualitätssicherung und Tests

7.1 Teststrategie

Die Teststrategie ist risikoorientiert. Vorrang haben Red-Flag-Erkennung, unmittelbare Eskalation, Dialogzustände, Datenimport, Vitalwerte und die strukturierte Ausgabe. UI-Details werden ergänzend manuell geprüft, da reine Controller-Tests nicht garantieren, dass Ereignisse im Browser zum richtigen Zeitpunkt ausgelöst werden. Die nachfolgende Tabelle beschreibt die verwendete Teststrategie genauer.

Ebene	Gegenstand	Beispiele
Unit-Test	Einzelne Fach- oder Hilfsfunktionen	Red-Flag-Regel, Scorekriterium, Sichtbarkeit einer Folgefrage, Simulatorbereich
Integrationstest	Zusammenspiel mehrerer Komponenten	Routinefall, Red-Flag-Fall, Abbruch, verweigerte Zustimmung
Manueller Teamtest	Tatsächlicher UI-Ablauf	Live-Eskalation, Simulatorintegration, Sichtbarkeit von Buttons, Sprachausgabe, sonstige Auffälligkeiten
Externer Usability-Test	Verständlichkeit und Bedienbarkeit	Von mindestens zwei Personen außerhalb des Teams durchgeführt

7.2 Automatisierte Testergebnisse

Die automatisierten Tests werden mit dem Framework pytest ausgeführt. Der Befehl `python -m pytest` durchläuft alle Testdateien im Verzeichnis `app/tests/unit/` und gibt eine Zusammenfassung der bestandenen und fehlgeschlagenen Tests aus. Die Tests werden bei jeder Code-Änderung lokal ausgeführt, um die Korrektheit der implementierten Logik zu überprüfen.

Aktualisiert am 02.07.2026:

`python -m pytest -q` – 271 Tests bestanden in 11,21 Sekunden.

Die verschiedenen Tests prüfen unter anderem:

- Routinefälle ohne Red-Flag und kritische Fälle mit Eskalation
- Die Red-Flags jedes Szenarios und die Unterscheidung von „warning“ und „critical“
- unmittelbare Eskalation aus Teilantworten sowie aus kritischen Gerätewerten
- Dialogzustände, Zustimmung, Ablehnung und Abbruch der Anamnese
- Patientenimport, Identitätsprüfung und drei Fehlversuche bei der Anmeldung
- bedingte Folgefragen und patientengerechte Sprache
- Blutdruck- und Gewichtsmessung bei unbekannten Werten
- Marburger-Herzscore-Kriterien
- Simulatorwerte und Wertebereiche
- JSON-/PDF-Export, Messwertquellen und strukturierte Zusammenfassungen

- F5-Browser-Refresh-Persistenz (`app.storage.user`)
- Exception-Handling in der UI-Hauptfunktion (`main_page()`)

7.3 Manuelle Prüfung und gefundene Fehler

Neben den automatisierten Tests wurden während der gesamten Entwicklungsphase regelmäßig manuelle Usability- und Systemtests durchgeführt. Diese erfolgten durch zwei Teammitglieder. Ziel war es, die Bedienbarkeit des Systems sowie die korrekte Umsetzung der funktionalen Anforderungen unter realistischen Nutzungsszenarien zu überprüfen.

Die umfangreichen Tests fanden mindestens zwei Mal pro Woche sowie zusätzlich nach größeren Implementierungsänderungen statt. Dabei wurde der vollständige Ablauf des Programms geprüft – von der Patientenidentifikation über die Szenarioauswahl und die Durchführung der Anamnese bis hin zur Simulation von Vitalparametern, der Red-Flag-Erkennung und der Erstellung der Zusammenfassung. Besonderes Augenmerk lag auf der korrekten Funktion sämtlicher Bedienelemente und Eingabemöglichkeiten. So wurden beispielsweise alle Auswahlmöglichkeiten von Dropdown-Menüs einzeln sowie in unterschiedlichen Kombinationen getestet, um fehlerhafte Zustandsübergänge oder inkonsistente Dialogabläufe aufzudecken.

Während der Tests wurden gefundene Fehler und Verbesserungsvorschläge dokumentiert und innerhalb des Entwicklungsteams über die gemeinsame WhatsApp-Gruppe kommuniziert. Anschließend wurden die gemeldeten Probleme behoben oder Verbesserungsvorschläge umgesetzt.

Kleinere Fehler, beispielsweise fehlende Button-Funktionen, konnten meist kurzfristig korrigiert werden. Komplexere Anpassungen betrafen unter anderem die Umstellung des ursprünglich schrittweisen Dialogs auf einen formularbasierten Anamneseablauf, bei dem alle Fragen gleichzeitig dargestellt werden. Diese Änderung erforderte mehrere Überarbeitungen, da durch die KI-gestützte Codegenerierung zunächst weitere Fehler und unerwartete Seiteneffekte entstanden.

Da größere Änderungen am Quellcode wiederholt Regressionen in bereits funktionierenden Bereichen verursachten, wurden nach umfangreichen Commits vollständige Systemtests erneut durchgeführt. Dadurch konnten neu entstandene Fehler frühzeitig erkannt und behoben werden. Die regelmäßigen manuellen Tests ergänzten somit die automatisierten Unit- und Integrationstests und leisteten einen wesentlichen Beitrag zur Stabilität und Benutzerfreundlichkeit des Gesamtsystems. Einige gefundene Fehler sind in den nachfolgenden Abschnitten beschrieben.

7.3.1 Gerätesimulator ohne fachlichen Auslöser

Ein Teammitglied implementierte einen Gerätesimulator. Bei der manuellen Prüfung durch ein anderes Teammitglied wurde festgestellt, dass der Simulator in keinem Szenario sichtbar ausgelöst wurde, die simulierten Werte jedoch trotzdem in der Zusammenfassung erschienen. Ursache war, dass die Simulation unbemerkt im Hintergrund gestartet wurde. Nach dem Hinweis wurde die Integration überarbeitet: Werte werden nun bewusst ausgelöst, szenariospezifisch übernommen und mit ihrer Quelle gekennzeichnet. Dieses Beispiel zeigt den Nutzen gegenseitiger Funktionstests über die selbst geschriebenen Unit-Tests hinaus.

7.3.2 Verzögerte statt unmittelbarer Eskalation

Codex meldete zunächst, dass die sofortige Eskalation für das Hypertonie-Szenario umgesetzt sei. Ein manueller Test zeigte jedoch, dass das Fragenformular nach einer kritischen Antwort weitergeführt wurde. Eine genauere Analyse ergab, dass der schrittweise Controller zwar nach jeder Antwort prüfte, das Formular seine Antworten aber erst beim Absenden übergab. Vorhandene Tests haben überwiegend den Controller abgedeckt und nicht das reale UI-Ereignis.

Daraufhin wurde die Red-Flag-Prüfung direkt an Auswahländerungen, das Verlassen von Freitextfeldern und simulierte Messwertänderungen gekoppelt. Ein kritischer Treffer beendet nun die Routinebefragung auch vor Formularabsendung (unmittelbare Eskalation). Ergänzt wurden zudem Tests für kritische Teilantworten und für kritische Simulationswerte in allen Szenarien.

7.3.3 Verpflichtende Blutdruckeingabe im Diabetes-Szenario

In der ersten Diabetes-Version waren systolischer und diastolischer Blutdruck als zwingend auszufüllende Zahlenfelder modelliert, die Möglichkeit den Wert als „unbekannt“ anzugeben, gab es nicht. Diese Einschränkung ist aus der erzeugten Implementierung entstanden. Nach Rückfrage wurde der Ablauf korrigiert: Patienten können nun „unbekannt“ auswählen, woraufhin anschließend eine Messung beziehungsweise Simulation dieser Werte angeboten wird. Tests prüfen sowohl den unbekannten als auch den bereits bekannten Wert. Leere Pflichtfelder bleiben weiterhin unzulässig und erzeugen beim Absenden des Formulars einen Hinweis, damit aktiv zwischen „vergessen“ und „unbekannt“ unterschieden werden kann.

7.3.4 Usability-Test mit externen Testpersonen

Im Rahmen einer explorativen Usability-Prüfung wurde der Prototyp zusätzlich mit zwei externen Testpersonen ausprobiert, die nicht an der Entwicklung beteiligt waren und keine Vorkenntnisse im Bereich Softwareentwicklung hatten. Die Testpersonen waren 68 und 53 Jahre alt. Ziel war eine erste Einschätzung der Verständlichkeit, Bedienbarkeit und Akzeptanz der Benutzeroberfläche.

Bei der älteren Testperson zeigte sich zunächst Unsicherheit gegenüber dem System. Die Person war skeptisch, wusste anfangs nicht eindeutig, welche Handlung als Nächstes erwartet wurde, und hatte Schwierigkeiten mit einzelnen UI-Elementen, insbesondere mit Slidern. Außerdem wurde die direkte Interaktion mit der KI als ungewohnt und teilweise unangenehm empfunden. Die Testperson bevorzugte deshalb das formularbasierte Ausfüllen der Anamnese, da dieses Vorgehen stärker an bekannte Abläufe in Arztpraxen erinnert. Auffällig war außerdem, dass Freitextfelder für ältere oder weniger tastaturgeübte Personen eine deutliche Hürde darstellen können. Das Tippen dauerte lange, und offene Eingabefelder führten teilweise zu sehr freien, kreativen oder irrelevanten Antworten. Daraus ergibt sich die Schlussfolgerung, dass möglichst viele Angaben über Auswahlfelder, Ja/Nein-Fragen oder klar geführte Eingabeelemente erhoben werden sollten. Freitext sollte nur dort eingesetzt werden, wo er fachlich wirklich notwendig ist.

Die jüngere externe Testperson zeigte sich offener gegenüber der Anwendung und konnte die Abläufe insgesamt gut nachvollziehen. Die sprachgeführte Anamnese wurde als interessant wahrgenommen. Dennoch war auch hier die direkte Interaktion mit der KI zunächst ungewohnt und musste vorgeführt werden. Dies deutet auf eine allgemeine Hemmschwelle hin, mit einem digitalen Assistenten aktiv zu sprechen, selbst wenn grundsätzlich Offenheit gegenüber der Technik besteht.

In beiden Testdurchläufen wurden keine Red-Flags ausgelöst. Die Beobachtungen zeigen jedoch, dass Bedienbarkeit und Akzeptanz besonders bei älteren Patientinnen und Patienten kritisch sind. Wenn bereits gesunde Testpersonen Schwierigkeiten mit der Bedienung haben, ist davon auszugehen, dass diese Hürden bei akuten Beschwerden, Stress oder eingeschränkter Konzentration noch stärker ins Gewicht fallen. Für eine Weiterentwicklung wären daher eine noch stärkere Vereinfachung der Oberfläche, größere und eindeutige Bedienelemente, weniger Freitext, eine bessere Einführung in die Sprachfunktion sowie ein klar sichtbarer formularbasierter Alternativweg sinnvoll.

8. Einsatz von KI im Projekt

8.1 Verwendete KI-Tools

Im Projekt sind zwei Arten von KI-Nutzung zu unterscheiden:

1. **KI als Entwicklungswerkzeug:** Codex, OpenCode und DeepSeek V4 Pro wurden je nach Präferenz des jeweiligen Teammitglieds für Analyse, Codegenerierung, Überarbeitung, Tests, Fehlersuche, Dokumentation und Diagrammentwürfe genutzt. Eine starre Zuordnung nach Aufgaben bestand nicht. Für umfangreiche Zusammenhänge wurde bevorzugt Codex mit GPT-5.5 High verwendet. Für kleinere Zusammenhänge kamen auch GPT-5.4 Medium/High zum Einsatz. DeepSeek wurde unter anderem für die Dokumentation, erste Diagramm-Entwürfe oder kleinere Programmieraufgaben verwendet.
2. **KI als optionale Prototypfunktion:** Der optionale Chat zur Schilderung der Beschwerden zu Beginn der Anamnese verwendet bei vorhandener Verbindung das lokale LLM der Hochschule (DeepSeek V4 Pro), um freie synthetische Texte in vorhandene Antwortfelder zu strukturieren. Red-Flags und Eskalation werden nicht durch dieses Modell entschieden.

8.2 Typischer KI-gestützter Arbeitsablauf

1. Aufgabenstellung, bestehende Dokumentation, Leitlinie und relevanter Code wurden der KI bereitgestellt.
2. Die KI wurde gebeten, zunächst Rückfragen zu stellen, um fachliche Absicht, Umfang und Integrationsweg zu klären.
3. Das Team entschied über Zielgruppe, Szenarioabgrenzung, Pflichtfelder, Folgefragen, Gerätepfade, Zusammenfassung und Sicherheitsgrenzen.
4. Die KI analysierte die vorhandene Architektur und schlug konkrete Änderungen vor.
5. Code und Tests wurden erzeugt oder überarbeitet.
6. Das Team las Ausgaben, verglich sie mit Aufgabenstellung und Leitlinien, startete Tests und prüfte zentrale Abläufe manuell.
7. Abweichungen wurden mit konkreten Beobachtungen zurückgemeldet. Die KI analysierte und korrigierte iterativ bis zum gewünschten Ergebnis.
8. Änderungen wurden über Git nachvollziehbar gespeichert und im Team zur Gegenprüfung angekündigt.

8.3 Beispiel-Prompts und Wirkung

8.3.1 Fachliche Überarbeitung eines Szenarios

Agiere als erfahrener Allgemeinmediziner. Analysiere den Diagnostik-Teil dieser NVL. Erstelle daraus einen leicht verständlichen Fragebogen für Patienten zur Erstanamnese im Wartezimmer. Verwende keine medizinischen Fachbegriffe. Bitte antworte im Chat und stelle mir zunächst Fragen zur Umsetzung. Es handelt sich um Szenario B „Brustschmerz in der Hausarztpraxis“. Enthält das PDF alle relevanten Informationen?

Der Prompt führte nicht unmittelbar zur Codegenerierung, sondern zunächst zu einer Quellen- und Umfangsdiskussion. Codex schlug eine alltagssprachliche Struktur vor, wies auf Grenzen der

Kurzfassung hin und identifizierte Fehler im vorhandenen Marburger-Herzscore. Erst nach Freigabe wurde die Implementierung geändert.

8.3.2 Aufbau eines neuen Diabetes-Szenarios

Es existieren bereits das chest pain und cough scenario. Ich würde gerne das diabetes scenario machen. Ich habe dir die Anforderungen und unsere bisherige Dokumentation hochgeladen. Bitte orientiere dich an den bisherigen Szenarien und stelle detailliert Fragen zur Umsetzung.

Die KI fragte nach fachlichem Zuschnitt, Verlaufskontrolle versus Akutfall, Komplikationen, Vorbefunden, Folgefragen, Red-Flags, Geräten, Zusammenfassung und Architektur. Das Team entschied unter anderem für eine ausschließlich Typ-2-Diabetes-Verlaufskontrolle, kontextabhängige Folgefragen, Blutdruck- und Gewichtserfassung, keinen Blutzuckersimulator, HbA1c als eigenes Feld und eine lesbar gruppierte Ausgabe. So entstand eine projektspezifische Lösung statt einer unkontrollierten Komplettgenerierung.

8.3.3 Überarbeitung vorhandener Szenarien einschließlich Tests

Vergleiche das vorhandene Szenario mit Aufgabenstellung und Leitlinie. Ergänze fehlende, patientengerechte Fragen und bedingte Folgefragen. Prüfe Red-Flags und Eskalation, überarbeite die strukturierte Zusammenfassung und ergänze passende Unit- und Integrationstests. Stelle vor der Umsetzung Rückfragen, wenn meine Absicht unklar ist.

Dieser Prompt wurde gemeinsam mit der Aufgabenstellung und den Leitlinien [1, 2, 3, 4, 5, 6] zur Überarbeitung der vier Szenarien verwendet. Die zugehörigen Commits änderten jeweils Szenariomodul, Regel-Engine, Dialogcontroller, SummaryBuilder und Tests. Die Tests umfassten sowohl unkritische als auch kritische Fälle sowie szenariospezifische Besonderheiten.

8.3.4 Fehlersuche anhand einer beobachtbaren Abweichung

Aber es kommt nicht zur sofortigen Eskalation. Der Fragebogen wird weitergeführt. Was bedeutet sofortige Eskalation?

Die präzise Beobachtung zwang die KI, ihre vorherige Erfolgsmeldung zu revidieren. Statt allgemein „funktioniert“ zu behaupten, wurde der Unterschied zwischen schrittweisem Controller und vollständigem Webformular analysiert. Daraus entstand eine UI-nahe Korrektur einschließlich neuer Regressionstests.

8.3.5 Hilfestellung zum Verstehen des Dialogmodells

Für unser Softwareprojekt, den KI-gestützten Anamnese-Agenten, müssen wir in der Dokumentation das Dialogmodell beschreiben. Was ist ein Dialogmodell? Und wie sieht es in

Da es im Code sehr schwer nachzuvollziehen war, wie genau das Dialogmodell aussieht, hat dieser Prompt an die KI geholfen, die Struktur des Dialogmodells besser nachzuvollziehen zu können. Nach einer kurzen Erklärung, was ein Dialogmodell ist, wurde übersichtlich dargestellt, wie das Dialogmodell in diesem Projekt aussieht. Die Zustände, Übergänge und die Dialogführungsstrategie wurden erklärt und teilweise grafisch dargestellt. Zudem wurden die wichtigsten Funktionen und relevanten Python-Dateien aufgezeigt.

8.4 Prüfung KI-generierten Codes

KI-Vorschläge wurden anhand mehrerer Kontrollquellen bewertet:

- Abgleich mit Muss-/Soll-/Kann-Anforderungen der Aufgabenstellung
- Abgleich medizinischer Inhalte mit den bereitgestellten Leitlinien
- Plausibilitätsprüfung von Fragen, Verzweigungen und Sicherheitsgrenzen
- Inspektion des geänderten Codes und seiner Einbindung in vorhandene Module
- automatisierte Unit- und Integrationstests
- manuelle Durchführung im Browser
- Gegenprüfung neuer Funktionen durch weitere Teammitglieder

Aus dem Gesprächsverlauf und den Git-Änderungen ist belegbar, dass Tests mit KI-Unterstützung erzeugt und ausgeführt wurden. Nicht vollständig rekonstruierbar ist jedoch, welche einzelnen Testfälle anschließend manuell ergänzt wurden.

8.5 Hilfreiche KI-Beiträge

- schneller Abgleich von Aufgabenstellung, Leitlinie und bestehendem Code
- strukturierte Rückfragen vor größeren Implementierungen
- Vorschläge für Folgefragen und lesbare Zusammenfassungsblöcke
- Erkennung einer fehlerhaften Marburger-Herzscore-Abbildung
- gleichzeitige Anpassung mehrerer zusammenhängender Module
- Erzeugung umfangreicher Regressionstests
- Analyse konkreter Tracebacks und Umgebungsprobleme
- Unterstützung bei README, Installation und Dokumentationsstruktur

8.6 Fehlerhafte oder riskante KI-Vorschläge

Vorschlag/Behauptung	Risiko	Erkennung	Korrektur
„Sofortige Eskalation ist umgesetzt“, obwohl das Formular erst beim Absenden prüfte	Kritischer Fall könnte unnötig weiterbefragt werden	Manueller UI-Test	Live-Prüfung an Eingabeereignisse und Gerätewerte; Regressionstests
Blutdruckwerte im Diabetes-Szenario als zwingende Zahlenfelder	Abbruch oder erfundene Eingabe bei unbekanntem Wert	Rückfrage gegen Aufgabenstellung	Option „unbekannt“ und anschließendes Mess-/Simulatorangebot
Erste Diabetes-Version ohne automatisierte Tests	Nur syntaktische, keine fachliche Absicherung	Expliziter Hinweis nach Implementierung	Spätere Szenario-, Regel-, Dialog- und Summary-Tests
Sprachmodus-Button sei sichtbar	Funktion praktisch nicht auffindbar	Manueller Browser-Test	Zusätzliche prominente Platzierung

Sprachausgabe nach Syntaxcheck als umgesetzt gemeldet	Browser-, Stimme- oder Berechtigungsproblem bleibt unentdeckt	Grenze der Entwicklungsumgebung	Praktische Prüfung im Zielbrowser und Tastatur-Fallback
Dynamische Einblendung von Simulatoren, wenn „unbekannt“ angegeben; „unbekannt“-Feld disabled wenn Wert simuliert wurde	„unbekannt“-Feld war nach Simulation immer noch enabled; simulierte Werte wurden nicht korrekt übernommen und nicht erkannt	Manueller UI-Test	Mehrfache Code-Anpassung und weitere manuelle Prüfung

8.7 Grenzen und Reflexion

Die KI beschleunigte Analyse und Implementierung, konnte jedoch den realen Browserablauf, akustische Ausgabe und fachliche Angemessenheit nicht eigenständig garantieren beziehungsweise überprüfen. Besonders problematisch waren zu weit gehende Erfolgsmeldungen nach isolierten Tests. Die wirksamste Arbeitsweise bestand daher aus kleinen, überprüfbaren Aufgaben beziehungsweise Prompts, klaren Sicherheitsgrenzen, Rückfragen vor der Umsetzung und einer anschließenden Kombination aus Codeprüfung, Tests und manueller Nutzung. Wurden die Aufgaben zu groß beziehungsweise wurde zu viel auf einmal verlangt, ist die KI manchmal an ihre Grenzen gestoßen. Einzelne Aspekte wurden zwar als umgesetzt bezeichnet, konnten in der Anwendung dann aber nicht gefunden werden. Auch verschiedene Funktionen (siehe Beispiele Tabelle oben, Abschnitt 8.6) konnten nicht immer mit dem gewünschten Ergebnis umgesetzt werden. Das ein oder andere Mal sind auch Funktionen, die vorher schon stabil funktioniert haben, durch eine Änderung verloren gegangen. Das hat den Arbeitsaufwand dann erhöht, weil mehrere Stellen angepasst und verändert werden mussten, bis das gewünschte Ergebnis erreicht war.

Insgesamt fand die Code-Generierung zu fast 100% KI-gesteuert statt. Da zu Beginn schon sehr große Codemengen generiert wurden, hat man schnell den Überblick verloren. Zudem generiert die KI Code mithilfe von Funktionen, die uns unbekannt sind. Dadurch ist das Projekt hinsichtlich des Codes sehr schnell sehr komplex geworden. Die Prüfung des KI-generierten Codes wurde zunehmend aufwendiger und schwieriger.

9. Projektmanagement und Teamarbeit

9.1 Projektorganisation

Zu Projektbeginn wurde die Aufgabenstellung in einer Excel-Übersicht in Kernprozess, funktionale und nichtfunktionale Anforderungen, medizinische Grenzen, Tests, Szenarien und Dokumentationspflichten zerlegt. Muss-, Soll- und Kann-Anforderungen wurden getrennt erfasst. Die Datei war zunächst als teaminterne Dokumentation von Zuständigkeit, Status (z.B. Aufgabe noch „offen“ oder „umgesetzt“) und Teststand vorgesehen. Im Projektverlauf wurde sie jedoch nicht konsequent fortgeschrieben und diente daher hauptsächlich als erste Orientierungs- und Priorisierungshilfe, um eine bessere Projektübersicht zu erhalten. Die laufende Koordination und Kommunikation erfolgte dann überwiegend mündlich an der Hochschule und über eine WhatsApp-Gruppe.

Git und GitHub dienten als gemeinsame technische Arbeitsgrundlage. Der Verlauf dokumentiert eine iterative Entwicklung: zunächst Architektur und Grundmodule, anschließend Anmeldung, Dialog, Szenarien, Red-Flags, UI, Exporte, Sprache, Personalmodus, Simulatoren, KI-Einbindung und die abschließenden Überarbeitungen der vier Szenarien. Fehlerkorrekturen wurden in eigenen Commits sichtbar. Die Qualitätssicherung erfolgte hauptsächlich über Tests, WhatsApp-Rückmeldungen und manuelle Gegenprüfung.

9.2 Aufgabenverteilung

Die laufende Aufgabenverteilung erfolgte flexibel, mündlich oder über WhatsApp. In diesem Projekt gab es keine dauerhaft abgegrenzten Rollen. Teammitglieder kündigten an, welche Aufgabe sie bearbeiten wollten. Nach einer Implementierung wurde entweder in der WhatsApp-Gruppe beschrieben, was geändert wurde, was beim Testen zu beachten war, sodass andere Mitglieder die Funktion gegenprüfen konnten, oder es mündlich mitgeteilt, was verändert oder hinzugefügt wurde. Letzteres fand meist montags nach der kurzen SET-Vorlesung statt, da hier das gesamte Team vor Ort war und man sich auf den neusten Stand bringen konnte. Zudem wurde in diesem Rahmen auch geklärt, welche Aufgaben in der kommenden Woche noch zu erledigen waren und wer diese übernimmt.

Obwohl zu Beginn keine festen Rollen verteilt wurden, hat sich auf diese Weise nach einiger Zeit eine gewisse Rollenverteilung ergeben beziehungsweise haben sich die einzelnen Teammitglieder selbstständig eigene Schwerpunkte gesetzt und jeder hatte am Ende ein Themengebiet, auf dem er sich besser auskennt als die anderen Teammitglieder. Üblicherweise schrieb die implementierende Person auch passende Unit-Tests. Eine koordinierende Person beschäftigte sich intensiver mit der Gesamtdokumentation, während eine andere viele manuelle UI-Tests durchführte und ein weiteres Teammitglied die gefundenen Auffälligkeiten anpasste und korrigierte.

9.3 Zusammenarbeit im Team

Obwohl es zu Beginn keine feste Rollenverteilung gab und der Start in das Projekt etwas unstrukturiert war, hat sich nach einer gewissen Zeit eine Struktur ergeben. Die Zusammenarbeit im Team lief sehr friedlich ab, jedes Teammitglied hatte die Möglichkeit, seine eigenen Stärken einzubringen und sich auf die Aufgabenfelder zu konzentrieren, die ihm mehr lagen beziehungsweise Spaß gemacht haben. So hat sich auf selbstständige Weise eine gewisse Rollenverteilung ergeben. Es

wurde regelmäßig miteinander kommuniziert, wenn neue Funktionen hinzugefügt wurden, Tests durchgeführt werden mussten oder es Probleme bei der Umsetzung bestimmter Aufgaben gab. Beispielsweise zeigt der Simulatorfehler den teaminternen Lernprozess: Die implementierende Person hatte eine Komponente technisch ergänzt. Eine andere Person testete den vollständigen Ablauf und erkannte, dass Werte ohne sichtbare Auslösung in die Zusammenfassung gelangten. Das Problem wurde kommuniziert und die Integration überarbeitet. Es wurde von keinem Teammitglied erwartet, dass jede Änderung am Code gleich einwandfrei funktionieren und testbar sein muss. Vielmehr durfte jeder im Team Stellen anpassen oder ergänzen, die ihm gerade aufgefallen waren. Durch die gewisse Rollenverteilung im späteren Verlauf gab es ein oder zwei Personen, die die Änderungen getestet und Auffälligkeiten gemeldet haben. Diese wurden dann selbstverständlich korrigiert.

Allgemein war die Arbeit und die Atmosphäre im Team sehr angenehm und hilfreich. Jedes Teammitglied hat die anderen so gut es ging unterstützt, wenn es einmal Probleme gab. Jeder konnte offen seine Meinung sagen, Verbesserungsvorschläge machen und wurde von den anderen Teammitgliedern respektiert und ernst genommen.

9.4 Herausforderungen im Projektverlauf

- Die anfängliche Excel-Übersicht wurde nicht als lebende Status- und Verantwortlichkeitsmatrix weitergeführt.
- Flexible Aufgabenwahl förderte Eigeninitiative, erschwerte aber die spätere eindeutige Beitragszuordnung.
- Informelle Reviews fanden statt, waren aber nicht einheitlich dokumentiert.
- KI-generierte Änderungen betrafen häufig mehrere Module und erhöhten den Review-Aufwand
- UI-Fehler wurden durch Controller-Tests nicht immer erkannt.
- KI-generierter Code ist oft sehr komplex

10. Installation und Bedienung

10.1 Voraussetzungen

- Python 3.12
- Git
- moderner Browser, vorzugsweise Chrome oder Edge für Sprachfunktionen
- optional Mikrofonaufberechtigung

10.2 Installation

Alternativ kann eine lokale virtuelle Umgebung verwendet werden. Entscheidend ist, dass die Anwendung mit dem Interpreter gestartet wird, in dem NiceGUI, ReportLab und die übrigen Abhängigkeiten installiert sind.

```
git clone https://github.com/Luisaaaaaaaaa/gruppe3.git
cd gruppe3
conda create -n SET python=3.12
conda activate SET
pip install -r requirements.txt
```

Um die KI-Funktionalitäten nutzen zu können, muss eine VPN-Verbindung mit dem Netzwerk der Hochschule bestehen.

10.3 Start

```
# Patientenmodus
python app/main.py          # Browser: http://127.0.0.1:8080

# Personalmodus
python app/main_personal.py  # Browser: http://127.0.0.1:8081
```

10.4 Optionaler KI-Chat

Für den Beschwerde-Chat wird eine Verbindung mit API-Schlüssel und Base URL hergestellt. Ohne Schlüssel oder Verbindung bleibt der strukturierte Fragebogen funktionsfähig. In den KI-Chat dürfen im Projekt ausschließlich synthetische Angaben eingegeben werden.

11. Einschränkungen und Ausblick

11.1 Bekannte Einschränkungen

- Reale Geräteadapter sind Platzhalter; es gibt keine Bluetooth-Integration oder echte Geräteanbindungen
- Simulatorverteilungen sind didaktisch und nicht klinisch validiert oder vollständig reproduzierbar konfiguriert
- Der optionale KI-Extraktor verwendet einen externen Dienst und ist nicht für reale Gesundheitsdaten freigegeben [7]
- Das Konsolenlogging einzelner Antworten ist für einen realen Datenschutzkontext zu umfangreich [7]
- Exporte besitzen noch kein Rollen-, Rechte-, Aufbewahrungs- und Löschkonzept
- Browser-Spracherkennung und Sprachausgabe sind browser- und systemabhängig
- Die medizinischen Regeln sind nicht klinisch validiert; der Prototyp ist kein Medizinprodukt [8]

11.2 Verbesserungsvorschläge für die Zukunft

- Mehrere Sprachen mittels KI ermöglichen - KI soll die Fragen in andere Sprachen korrekt übersetzen und ausgeben
- Anbindung an Patientenkarte, damit auf die digitale Patientenakte und Personendaten zugegriffen werden kann
- Kompletter Chat-Verlauf KI geführt
- Weitere Szenarien umsetzen
- Echte Geräte-Schnittstellen einrichten
- Erklärung der Sprachfunktion des Systems
- Interaktion mit dem System noch einfacher gestalten – weniger Freitextfelder, mehr Buttons oder vorgegebene Auswahlmöglichkeiten

11.2 Ausblick auf eine Praxisintegration

Eine mögliche Weiterentwicklung beginnt mit URL-Import einschließlich Schema-Validierung, Timeouts und verständlicher Fehlerbehandlung. Danach sollten fachliche Konfigurationen versioniert, Geräte über klar definierte Schnittstellen angebunden und Simulatoren deterministisch testbar gemacht werden. Für die Integration in Praxissoftware wäre eine standardisierte Schnittstelle, perspektivisch, beispielsweise FHIR, sinnvoll.

Vor jeder realen Nutzung wären Datenschutz-Folgenabschätzung, Rollen- und Rechtekonzept, Verschlüsselung, Protokollminimierung, sichere Schlüsselverwaltung, regulatorische Einordnung, klinische Risikoanalyse und fachliche Validierung erforderlich [7, 8]. Ergänzend sollten Barrierefreiheit, Mehrsprachigkeit, robuste lokale Spracheingabe und systematische Usability-Studien untersucht werden.

12. Fazit

Im Projekt entstand ein modularer Prototyp, der vier hausärztliche Szenarien strukturiert abbildet, Patientenvorinformationen einbezieht, Folgefragen stellt, Vitalwerte mit Herkunft erfasst, deterministische Red-Flags prüft und eine gegliederte Übergabe erzeugt. Zum Zeitpunkt der Projektdokumentation wurden 236 automatisierte Tests erfolgreich ausgeführt. Die Ergebnisse belegen den aktuellen Entwicklungsstand des Systems, stellen jedoch keinen Nachweis vollständiger Fehlerfreiheit dar. Besonders wertvoll war die Kombination aus KI-gestützter Entwicklung und menschlicher Gegenprüfung.

Die dokumentierten Fehler zeigen zugleich die Grenzen dieses Vorgehens. KI-generierter Code konnte syntaktisch korrekt und isoliert getestet sein, obwohl der tatsächliche UI-Ablauf eine Sicherheitsanforderung noch nicht erfüllte. Erst manuelle Tests, Rückfragen gegen die Aufgabenstellung und teaminterne Prüfung machten diese Abweichungen sichtbar. Der wesentliche Lernerfolg liegt daher nicht allein in der erzeugten Software, sondern in einem kontrollierten Entwicklungsprozess: Anforderungen präzisieren, kleine Änderungen erzeugen, fachlich prüfen, automatisiert testen, praktisch erproben und transparent dokumentieren.

Allgemein war die Arbeit an diesem Projekt sehr lehrreich und interessant, wenn auch sehr zeitintensiv und umfangreich. Obwohl wir unstrukturiert in das Projekt gestartet sind und es keine eindeutige Rollenverteilung gab, sind wir mit dem Ergebnis dieses Projekts sehr zufrieden. Dass es noch bekannte Einschränkungen gibt, die oben bereits beschrieben wurden, ist uns bewusst. Die Erwartung war nicht, dass das Projekt zum Schluss perfekt läuft und alles wie gefordert umgesetzt wurde. Vielmehr darf das Projekt an der ein oder anderen Stelle noch unvollständig sein und Raum für Verbesserungen besitzen. Ziel des Projekts war es primär, den Umgang mit KI als Code-Generierungstool zu erlernen, den Einsatz von KI bewusst zu nutzen und ihn kritisch zu hinterfragen und überprüfen. Dieses Projekt ist eine Teamarbeit und auch die Arbeit im Team muss erst gelernt werden. In dieser Hinsicht haben wir als Team viele Erfahrungen gesammelt, wissen jetzt, was gut funktioniert und woran wir noch arbeiten müssen.

Abschließend können wir ein positives Fazit aus dieser Projektarbeit ziehen. Unser KI-gestützter Anamnese-Agent besitzt noch Verbesserungspotential und somit besteht die Möglichkeit, dieses Projekt eventuell in Zukunft noch einmal aufzugreifen und weiter daran zu arbeiten.

13. Literaturverzeichnis

1. Vetter, M.: *SET-Semesterprojekt Sommersemester 2026 – KI-gestützter Anamnese-Agent für die hausärztliche Praxis*. Aufgabenstellung, Version 1.0, 12.05.2026.
2. Deutsche Gesellschaft für Allgemeinmedizin und Familienmedizin (DEGAM): *Husten*. DEGAM-Leitlinie, Kurzversion, Version 3.0, AWMF-Register-Nr. 053-013, 2021.
3. Deutsche Gesellschaft für Pneumologie und Beatmungsmedizin u. a.: *Behandlung von erwachsenen Patienten mit ambulant erworbener Pneumonie*. S3-Leitlinie, AWMF-Register-Nr. 020-020.
4. Bösner, S.; Donner-Banzhoff, N.; Escales, C.; Haasenritter, J.: *Brustschmerz*. DEGAM-Leitlinie, Kurzversion, Version 2.2, AWMF-Register-Nr. 053-023. Deutsche Gesellschaft für Allgemeinmedizin und Familienmedizin (DEGAM), 2024; Überarbeitung 06/2024.
5. Bundesärztekammer; Kassenärztliche Bundesvereinigung; Arbeitsgemeinschaft der Wissenschaftlichen Medizinischen Fachgesellschaften: *Nationale VersorgungsLeitlinie Hypertonie – Kurzfassung*. Version 1.0, AWMF-Register-Nr. nvl-009, 2023.
6. Bundesärztekammer; Kassenärztliche Bundesvereinigung; Arbeitsgemeinschaft der Wissenschaftlichen Medizinischen Fachgesellschaften: *Nationale VersorgungsLeitlinie Typ-2-Diabetes – Kurzfassung*. Version 3.0, AWMF-Register-Nr. nvl-001, 2023.
7. Europäische Union: Verordnung (EU) 2016/679 (Datenschutz-Grundverordnung).
8. Europäische Union: Verordnung (EU) 2017/745 über Medizinprodukte.
9. Projektteam 3: Quellcode-Repository *gruppe3*, Repository-Stand 21.06.2026.